第九届传智杯 · AI Web 赛道 · B 组 · 省赛作品

芯诊 AI

证据驱动的 STM32 USART 故障诊断工作台

技术文档（省赛提交材料）

团队成员：李兴明 · 王永康（队长）· 陈璇

版本：v1.1 · 2026-10-26

形态：Web 应用 · 规则层 + 确定性工具 + RAG + 本地大模型解释

可追溯 · 不硬猜 · 可重复 · 证据纪律

目录

一、产品概述

二、需求分析

三、系统架构

四、AI 方案设计

五、关键技术实现

六、测试与评测

七、部署与运行

八、创新点与社会价值

九、总结与展望

附录 A：代码目录速查

附录 B：术语与证据标签说明

芯诊 AI · 技术文档

> 作品名称：芯诊 AI —— 证据驱动的 STM32 USART 故障诊断工作台
> 参赛赛事：第九届传智杯 · 赛道：AI Web（B 组）
> 团队成员：李兴明 · 王永康（队长）· 陈璇
> 文档版本：v1.1（2026-10-26）
> 适用读者：评委 / 开发者 / 使用者

文档信息

项目	内容
作品名称	芯诊 AI：证据驱动的 STM32 USART 故障诊断 Web 工作台
参赛赛事	第九届传智杯 · AI Web 赛道（B 组，Web 应用）
团队成员	李兴明、王永康（队长）、陈璇
文档版本	v1.1（2026-10-26）
交付形态	前端 React Web 应用为主体，后端 FastAPI 作 AI 接入层
关键词	STM32 故障诊断 · 证据驱动 · 多步验证闭环 · 规则引擎 · RAG 检索 · 本地大模型 · 云端可切换

版本记录

版本	日期	说明
v1.0	2026-10-25	基于 Phase 0–7 软件侧交付内容整理（初版）
v1.1	2026-10-26	按赛事提交材料规范重构结构与排版（文档信息/摘要/目录/附录；正文事实与数据不变）

摘要

芯诊 AI 是一款面向 STM32 嵌入式开发者的**证据驱动故障诊断 Web 工作台**。针对"串口不通、乱码、无输出"等高频 USART 故障，用户粘贴代码片段、编译日志或串口抓帧后，系统以 8 个确定性诊断工具提取代码事实，经 23 条结构化规则推理与自建 40 张知识卡片检索，输出**带证据链的候选根因**与**可上机执行的验证步骤**；用户在真机回填实测结果后，系统按证据重排结论并出具七章结构化诊断报告。

项目以"可追溯、不硬猜、可重复"为核心价值：每条结论可点回证据，材料不足时明确拒绝猜测，同一证据得到确定性结论。AI 采用"规则层 + 确定性工具 + RAG + 大模型解释"的权责分离架构，模型恒标 [推断]、无诊断权，输出经守卫校验并支持三级降级（云端 DeepSeek → 本地 Ollama → 规则层），断网亦可完整演示。软件侧 8 套件自动化测试全绿（工具 52 / 样例 24 / 模型层 45 / 评测 9 / 边界 16 / 安全审计 13 / 端到端 76 / 模型冒烟 16），固定评测集正例 Top-1 命中 6/6、负例 0 误报。

一、产品概述

1.1 项目背景

嵌入式调试中，"串口不通"的可能原因成百上千：外设时钟未使能、引脚模式配错、波特率对不上、晶振频率与实际不符……传统做法是拿代码问大模型，但它常常自信地给出**无法验证的结论**。本作品用工程化思路解决这一问题：把"猜答案"变成"查证据"。

1.2 产品定位

一个以**诊断任务为中心**的 Web 工作台（非聊天框）：用户提交故障材料，系统用确定性工具提取证据、检索自建知识库、按规则推理出带证据链的候选根因，并给出可在真机上执行的验证步骤；用户回填实测结果后，系统重排结论并出具结构化诊断报告。

1.3 核心价值

价值	含义
可追溯	每条结论都能点回证据（源码行、寄存器配置、波特率计算、抓帧统计）
不硬猜	材料不足时明确说"不足"并列出待补材料，不给无法验证的猜测
可重复	同一任务重跑、同一证据得到同一结论（确定性工具，无随机）

1.4 核心能力

能力	说明
证据驱动诊断	23 条 UART 结构化规则 + 8 个确定性工具 + BM25 检索 40 张自撰知识卡片，端到端留痕
多步验证工作流	候选根因 → 验证步骤 → 用户在真机回填结果 → 系统重排 → 结构化报告
可追溯报告	结论挂证据与工具调用记录；重新生成保留历史版本
模型安全可控	三级降级链（云端 DeepSeek → 本地 Ollama → 规则层）、输出守卫、模型无诊断权
零部署依赖	SQLite 单文件 + 本地服务，clone 即跑；密钥零入库（.env.example）

二、需求分析

2.1 目标用户

嵌入式初学者、高校实训学生、维护一线工程师——在遇到"串口不通/乱码/无输出"类故障时，需要一个**能解释为什么**的诊断工具，而不是一个可能自信地给出错误结论的聊天框。

2.2 功能需求（九步主流程）

1. 新建诊断任务，粘贴源码（+ 可选编译日志、实测波特率、串口抓帧）；
2. 描述故障现象；

3. 系统判定材料是否充分（无 USART 证据 → 明确"材料不足"并列出待补清单）；
4. 8 个确定性工具依次执行并留痕；
5. 规则匹配 + 症状加权 → 候选根因排序（含参考卡片）；
6. 每个候选生成可执行的验证步骤（预期现象）；
7. SSE 实时推送全过程；
8. 用户回填实测结果 → 系统重排候选；
9. 生成七章结构化诊断报告（可重新生成、保留版本）。

2.3 非功能需求

- **可追溯**：所有结论挂证据与工具调用记录；
- **可控**：模型输出恒 [推断]、有输出守卫、失败降级不中断；
- **安全**：不执行用户代码、密钥零入库、无 SQL 注入 / XSS 面；
- **零部署成本**：SQLite + 本地服务，提交即可运行；
- **可用性**：空 / 加载 / 错误 / 恢复四态齐全，演示断网可降级。

2.4 MVP 边界（明确排除）

- 仅 STM32F103C8T6 + USART + 标准库；
- 不做多芯片 / 多协议扩展；不做多模态输入；不做云端推理依赖；
- 排除项与理由见项目内部文档「需求与架构 → 产品需求文档」（公开仓库不含）。

三、系统架构

3.1 总体分层



3.2 数据流（一次诊断）

1. 用户提交 `source_code` + `symptom_text`（+ 可选 `build_log`、波特率实测值）；
2. 引擎做**材料完整性前置判定**：无 USART 证据 → 直接 `unknown`，列待补材料；
3. 有证据 → 8 工具依次提取事实（时钟使能 / GPIO 模式 / 波特率误差 / ISR 体 / 抓帧统计…）；
4. L1 规则匹配 + 症状加权 → 候选根因排序；BM25 检索挂参考卡片；
5. 每个候选生成**验证步骤**（哪些能在真机做、预期现象是什么）；
6. SSE 实时推送：任务状态 / 工具调用 / 候选更新 / 证据落库；
7. 用户在真机执行验证并回填 `observed` / `match`；
8. 引擎按回填重排候选，生成七章报告（含硬件实测记录章，全部带证据标签）。

3.3 关键设计决策（ADR 摘要）

决策	理由
规则层为先、模型只解释	确定性工具不产生幻觉；模型负责语言表达，二者权责分离

自研状态机而非 LangChain	编排逻辑简单、可测试、零依赖
BM25 中文 2-gram + ASCII 整词	自建语料仅 40 条，词级切分命中差；评测中参考命中 45 次
SQLite + 本地文件	零部署、提交即跑
三级降级链	无网 / 无模型时功能不塌，演示容错
证据标签入类型系统 + DB CHECK	在编译期和存储层双重杜绝"把推断写成实测"

四、AI 方案设计

4.1 设计思路：权责分离

系统不依赖大模型"凭空诊断"，而是采用**规则层 + 确定性工具 + RAG + 模型解释**的四层权责分离：

- **确定性工具**（8 个）负责查证：从代码 / 日志 / 抓帧中提取可验证事实；
- **规则层**（23 条）负责判定：对事实做结构化匹配，输出 violated / ok / unknown 三态；
- **RAG**（40 张自建卡片）负责知识引用：为结论挂可追溯的参考卡片；
- **大模型**（本地 Ollama / 云端 DeepSeek）仅负责**语言表达**：把已查出的结论解释成大白话、补充验证建议；恒标 [推断]，无诊断权，不参与根因判定与评分。

4.2 模型选型理由

为什么本地小模型 + 规则层，而不是只做大模型：

维度	纯 LLM 方案	芯诊 AI（规则层+工具+RAG+LLM 解释）
材料不足	可能硬猜根因（评测 3 负例中踩坑）	明确"材料不足"并列出待补清单（0 误报）
超范围材料	可能把 I2C 硬套成 UART 结论	识别范围外，拒绝套结论
可追溯	无证据链	每条结论可点回证据
稳定性	输出抖动（同输入多次结果不同）	确定性工具，同证据同结论
承载实测	无法产生 [实测-硬件]	T-07 抓帧回填通道，实测由用户产生
成本 / 隐私	云端 API 依赖网络与密钥	本地运行、零密钥、断网可演示；云端（DeepSeek）为可选增强

本地模型三模型基线横向对比（固定评测集 9 用例：6 正 + 3 负，同一批材料、同一候选空间、每模型 3 次采样，趋势口径）：

指标	规则层+工具	qwen2.5:7b	qwen2.5-coder:7b	qwen2.5:3b
正例 Top-1 全中	6/6	6/6	6/6	5/6
负例误报（任一采样）	0	2	0	1

输出稳定（×3 一致）	确定性	7/9	9/9	8/9
平均单次耗时	<10 ms	2736 ms	2247 ms	895 ms

选型结论：默认模型为 **qwen2.5-coder:7b**（正例不输、负例防坑最好、输出最稳、答对唯一的编译错误用例）；qwen2.5:3b 保留为弱机 / 快速备选；qwen2.5:7b 作备选。

> ⚠️ 口径声明：样本仅 9 例，只作趋势；**不宣称"比大模型更准"**。真实差异是可追溯、不硬猜、超范围不套结论、能承载 [实测-硬件]。

4.3 模型集成方式

- **接入**：标准库 `urllib` 直连 Ollama HTTP API 与云端 OpenAI 兼容 `/chat/completions`（**零第三方依赖**）；
- **降级链**：云端（可选 8 家服务商）→ 本地 Ollama → 规则层，按 `XZ_LLM_PREFERRED`（auto / local / cloud）决定来源顺序，健康检查带缓存（20s），失败自动降级并留痕；
- **鉴权**：云端走 `Bearer`，健康探测 `GET /models` 不触发计费；
- **提示词**：分层设计——系统提示（角色 + 证据纪律）、任务提示（故障类型）、上下文注入（代码片段 + 日志 + 检索结果）；
- **稳定性**：输出守卫截断与校验；温度 0.2 抑制发散；冷启动靠健康缓存与演示预热规避。

4.3.1 多服务商（云端不绑定某一家）

云端按**服务端白名单**支持多家 OpenAI 兼容服务商：前端下拉选择，请求头 `X-Cloud-Provider` 只传 id，`base_url` 由后端预设表解析——**绝不接受请求方传入地址**（否则服务即成为任意地址的代理，SSRF）。

服务商	预设地址（服务端白名单）	默认模型
DeepSeek（深度求索）	`api.deepseek.com`	`deepseek-chat`
通义千问（DashScope）	`dashscope.aliyuncs.com/compatible-mode/v1`	`qwen-plus`
Kimi（月之暗面）	`api.moonshot.cn/v1`	`moonshot-v1-8k`
智谱 GLM	`open.bigmodel.cn/api/paas/v4`	`glm-4-flash`
硅基流动 SiliconFlow	`api.siliconflow.cn/v1`	`Qwen/Qwen2.5-7B-Instruct`
OpenAI	`api.openai.com/v1`	`gpt-4o-mini`
OpenRouter（聚合）	`openrouter.ai/api/v1`	`deepseek/deepseek-chat`
Groq	`api.groq.com/openai/v1`	`llama-3.3-70b-versatile`

模型名允许自填（各家迭代快），仅做字符白名单校验；密钥仍由测试者自带、只在请求内存内使用。

4.4 输出守卫与降级链

- 结构校验：JSON / 字段类型（summary / extra_steps / cautions 白名单）；
- 长度截断：摘要 400 字符、验证步骤 4 条 × 120 字符；

- 校验失败 → 降级规则层并记录原因；
- 模型不可用 / 超时 / 云端密钥未配置 → 自动降级，不中断演示。

4.5 AI 安全与可控

- **模型无诊断权**：不参与根因判定、不改评分、不新增候选；
- **恒 [推断]**：模型输出一律标记推断，UI 与报告端到端强制；
- **本地优先 / 云端可选**：默认 auto——云端就绪则优先，否则回落本地；密钥留空云端自动跳过，绝无"半配置"运行；前端可切换「自动 / 本地 / 云端」。
- **密钥由测试者自带**：云端调用使用访客自己的 API Key。密钥只存浏览器 `sessionStorage`（关闭标签页即失效），仅随请求头 `X-Cloud-Key` 发到后端内存，**用完即丢：不写数据库、不写文件、不写日志、不进 URL**（避免被访问日志记录）；任何响应只返回掩码（`sk-ab…cd90`）。界面在顶部常驻"测试结束请删除密钥"提示，并提供一键删除；服务端 `POST /api/llm/verify-key` 只做一次鉴权探测、不做推理计费。服务端环境变量密钥仅作为站长自用兜底。

五、关键技术实现

5.1 证据纪律体系（六档证据标签）

标签	含义	谁能产生
[实测-硬件]	用户上板回填的原始记录	仅用户（T-07 抓帧回填通道）
[实测-仿真]	Keil 仿真 / 模拟环境运行结果	预留
[静态分析]	代码 / 配置 / 日志静态检查	工具层（T-01…T-06）
[推断]	模型基于证据的推理	仅模型（恒带此标签）
[假设]	无证据支撑的猜测	系统不产出（语料中知识等级标注）
[待验证]	计划中未执行	阈值 / 现象对照表

落地机制：Python 类型系统构造校验 → 接口层证据标签白名单 → SQLite CHECK 约束 → 报告渲染器将标签渲染为徽章。`_tag()` 对非法标签直接抛异常，不静默降级。

5.2 确定性诊断工具（8 个）

编号	工具	输入 → 事实	证据等级
T-01	parse_build_log	编译/链接日志 → 错误行、缺定义	[静态分析]
T-02	analyze_c_code	C 源码 → 外设/引脚/时钟调用事实	[静态分析]
T-03	check_uart_config	源码 → UART/GPIO/RCC/NVIC 一致性	[静态分析]
T-04	calc_baud_error	配置 → 波特率误差%（阈值 2%，[待验证]）	[静态分析]
T-04r	infer_actual_fck	实测波特率 → 反推实际总线时钟	[推断]

T-05	search_reference	症状/证据 → BM25 相关卡片	[静态分析]
T-06	analyze_isr_body	ISR 体 → 清标志/进/出中断检查	[静态分析]
T-07	parse_serial_capture	抓帧 → EMPTY/GARBLED/NORMAL 统计	唯一可产 [实测-硬件]
T-08	llm_assist（按需）	证据摘要 → 大白话解释 + 补充验证建议	恒 [推断]

5.3 自建语料与轻量 RAG（全部自撰，无版权风险）

- **L1 规则库**：docs/corpus/l1_rules_uart.json，23 条，覆盖 6 类故障场景 + 通用规则；
- **L2 知识卡片**：docs/corpus/cards/，40 条 × 6 主题（时钟/波特率/GPIO/重映射/NVIC/printf）；
- **现象对照表**：phenomena_map.json，6 条，全部标 [推断]，待硬件实测回填校准；
- **检索**：BM25（中文 2-gram 切分 + ASCII 整词），语料小、可追溯；
- **知识来源**：公有技术事实（芯片手册章节级引用，非原文转载）+ 自撰工程经验；
- **消融结论**：去掉 RAG → 命中率不变但参考 45→0（RAG 贡献 = 可追溯性，非准确率）。

5.4 应用安全（security_audit 13 项全过）

- 不执行用户代码：诊断工具纯静态解析；
- 无 SQL 注入面：SQLite 访问全部参数化；
- 密钥零入库：.env 排除、扫描器验证无泄漏；
- XSS：React 默认转义 + 报告为受控 Markdown 子集渲染；
- 上传限制：单文件解析上限 512 KB。

六、测试与评测

6.1 自动化测试体系（8 套件一键回归）

命令：`cd apps/server && python run_all_tests.py`（子脚本退出码非 0 即 FAIL；依赖不可达判 SKIP 不判通过）。

套件	数量	结果
工具层单测（8 工具确定性行为）	52	✅ 52/52
故障样例自检（12 个自撰样例）	24	✅ 24/24
模型层测试（守卫/降级/在线真实调用，含云端切换 + 自带密钥 + 多服务商）	50	✅ 50/50
固定评测集（6 正 + 3 负）	9	✅ 判定通过
边界与异常（超大/空输入/非法标签/注入）	16	✅ 16/16
仓库合规审计（密钥/版权/XSS）	13	✅ 13/13（1 条正当 WARN）

端到端 HTTP 冒烟	80	✅ 80/80
模型解读链路冒烟	16	✅ 16/16

6.2 固定评测集与指标

- 6 正例：C1 时钟未使能 / C2 波特率不符 / C3 TX 引脚模式 / C4 重映射无 AFIO / C5 NVIC 未挂接 / C6 printf 未重定向；
- 3 负例：N-01 材料严重不足 / N-02 编译错误被描述成硬件故障 / N-03 I2C 超范围硬套；
- 指标：Top-1 命中、负例误报、证据可追溯、自动幻觉筛查。

指标	结果
正例 Top-1 命中	6/6
负例误报	0/3
证据可追溯	8/8
自动幻觉筛查	0 条可疑

评测抓出并修复的真缺陷（诚实记录）： ① 空材料硬给根因 → 新增材料前置判定；② I2C 被硬套 UART 结论 → 编排层超范围 followup；③ 材料不足缺引导 → 待补清单；④ 幻觉筛查器自身 2 处误报 → 修正匹配逻辑。

6.3 消融实验

实验	结果	结论
去掉 RAG	命中率不变，参考 45→0	RAG 贡献 = 可追溯，非准确率
去掉症状加权	0/9 排序改变	当前测试集影响为 0，不宣称"症状驱动排序"
关键词朴素基线	3 负例误报 2	复现 N-01/N-03 两个坑

七、部署与运行

7.1 环境要求

- Python 3.11+（开发用 3.13）；Node.js 18+（开发用 22.x）；
- （可选）本地 Ollama http://127.0.0.1:11434，已拉取 qwen2.5-coder:7b（默认）等；
- 没有模型也不影响：自动降级纯规则层。

7.2 启动步骤

```
bash
# 后端
cd apps/server
python -m venv .venv && .venv/Scripts/activate
pip install fastapi "uvicorn[standard]" pydantic
python -m uvicorn app.main:app --host 127.0.0.1 --port 8787

# 前端（另开终端）
cd apps/web
npm install
npm run dev          # http://127.0.0.1:5173

# 一键回归
cd apps/server && python run_all_tests.py
```

7.3 配置说明

阈值与服务端兜底密钥走环境变量：apps/server/.env.example（XZ_* 前缀，与 config.py 一致）。云端默认 DeepSeek（XZ_CLOUD_MODEL=deepseek-chat，备选 deepseek-reasoner）。**正式演示不需要站长填任何密钥**：测试者在页面「我的云端密钥」里填自己的 Key 即可（只存本标签页、随请求头传递、服务端不保存），测试结束点「删除密钥」或关掉标签页即清除。

7.4 云服务器部署（子路径 • 云端 AI 为主）

线上地址：https://zerovoidlog.cn/xz/（已部署于阿里云 ECS，与作者博客 `zerovoidlog.cn` 共存：Nginx 以 `/xz/` 前缀提供静态前端与 API 反代，共用域名、证书与备案）

- 形态：`https://域名/xz/` 静态前端 + `/xz/api/` 反代 `127.0.0.1:8787`（systemd 常驻、单 worker），与现有站点共存，共用域名与证书，不新增备案；
- 前端按子路径构建：`VITE_BASE=/xz/ npx vite build`（资源路径随之改为 `/xz/assets/...`；API 前缀取 `import.meta.env.BASE_URL`）；
- Nginx 关键项：`proxy_buffering off`（SSE 流式必需）、`proxy_read_timeout 300s`、`client_max_body_size 5m`；
- 模型：服务器无 GPU 时不跑本地 7B，`XZ_LLM_PREFERRED=cloud` 走云端；本地层探测失败自动跳过，不拖慢诊断；
- 部署件见 `deploy/`（systemd unit / Nginx 片段 / 生产环境变量 / 逐步说明）。

7.4 零依赖与断网降级

本地模型不可用或网络断开时，系统自动降级为纯规则层，核心诊断能力不中断，适合现场演示与教学场景。

八、创新点与社会价值

8.1 技术创新点

1. **证据纪律系统化**：六档证据标签从 Python 类型系统、接口白名单到 SQLite CHECK 约束三层落地；模型恒 [推断]、无诊断权——AI 输出安全与可控性内建在架构里，而非提示词约束；
2. **规则层 + 工具 + 模型权责分离**：确定性工具查证、规则层判定、模型只解释，模型幻觉被结构性隔离；
3. **多步验证闭环**：候选 → 验证步骤 → 真机回填 → 重排 → 报告，诊断是可复核的实验过程而非一次问答；
4. **自研轻量 LLM 接入**：标准库直连 Ollama 与 DeepSeek、零第三方依赖、三级降级链（云端→本地→规则层，可按偏好切换来源），断网也能完整演示；
5. **自建领域语料 + 轻量 RAG**：23 规则 + 40 卡片全部自撰（零版权风险），BM25 中文 2-gram 检索，贡献可追溯；
6. **材料不足识别**：把"没看到"与"有错"区分开，评测负例 0 误报。

8.2 应用创新点

1. 以「诊断任务」为中心的证据面板工作台（非聊天框）：候选卡片、支持 / 反对证据、可点击验证步骤、实测回填后自动更新判断；
2. 硬件实测证据由用户上板产生、系统承载并重排——AI 不越权"宣布实测"；
3. 降低嵌入式调试门槛：高校实训与一线排障的"师傅级"证据链可复制。

8.3 社会价值与商业潜力

- **教学价值**：为高校实训提供"可解释的故障诊断教具"，初学者能看懂每一步判定依据；
- **工程价值**：沉淀团队调试经验为可检索、可追溯的知识资产；
- **商业潜力**：面向嵌入式培训、产线排障辅助工具场景，具备低成本本地化部署优势。

九、总结与展望

9.1 项目总结

本项目用"确定性工具查证 + 规则推理 + 模型解释"的组合，把 STM32 USART 故障诊断从"猜答案"变成"查证据"，形成可追溯、不硬猜、可重复的闭环。软件侧 8 套件全绿、9 用例评测达标、安全合规通过、默认模型经三模型基线选定，并支持本地 Ollama ↔ 云端 DeepSeek 一键切换（密钥零入库）。

9.2 展望

- **硬件实测回填**（后续扩展）：T-07 抓帧回填通道与上板方案（C1–C6）已就绪并保留，可产生系统首批 [实测-硬件] 证据并校准阈值；本期按范围决策不执行；
- **协议扩展**：USART2/3、SPI、I2C、CAN 的规则库与工具逐步扩展（MVP 边界内保持不硬套）；
- **多模态**：串口抓帧波形图、示波器截图识别（当前 P1-D 决策未投入，作为后续方向）；
- **云端增强**：DeepSeek 已接入降级链第①级（XZ_CLOUD_API_KEY 填入即启用，前端可切换来源；deepseek-reasoner 备选），后续可接入更多 provider；

- **在线演示**：部署公网版本供评审访问（可选加分项）。

附录 A：代码目录速查

```
apps/server/app/engine/     8 工具 + orchestrator
apps/server/app/corpus/     规则/卡片加载 + BM25
apps/server/app/llm/        客户端 / 提示词 / 守卫 / 降级
apps/server/app/            main.py（路由）/ db.py（13 表）
apps/server/eval/           评测集 / 消融 / 边界 / 安全 / LLM 基线 / run_all_tests.py
apps/server/samples/        12 个自撰样例
apps/web/src/               前端（components/ 三栏 + 报告渲染器）
docs/                       方案 / 验收 / 语料 / 风险 / 决策记录
deliverables/               对外交付材料
firmware/baseline_usart1/   基准工程（自撰）
```

附录 B：术语与证据标签说明

术语 / 标签	说明
证据标签	系统内每个结论必须携带 [实测-硬件] / [实测-仿真] / [静态分析] / [推断] / [假设] / [待验证] 之一
[实测-硬件]	仅用户上板回填可产生；系统内当前 0 条（本期范围决策不纳入）
[推断]	模型输出恒带；仅作解释与验证建议，无诊断权
候选根因	按证据支持度排序的故障原因列表，每条挂证据链
验证计划	每个候选配的可执行步骤（引脚 / 预期现象），用户回填后触发重排
七章报告	诊断报告结构：任务信息 / 现象 / 证据 / 候选 / 无法判定 / 知识卡片 / 硬件实测记录
RAG	检索增强生成，本项目用于知识卡片检索，贡献可追溯性
三级降级链	云端 DeepSeek → 本地 Ollama → 规则层，保证断网可演示